Application & Data Security (CS4021) — Assignment 1

Threat Modeling, Vulnerability Exploitation, and Mitigation (OWASP Top 10)

Student Name: Aditya Ranjan Kumar
Roll Number: CS-2341901
Class & Section: 4CSE9
Course Instructor: Dr. Ajay Prasad
Submission Date: 15-10-2026
GitHub Repository: https://github.com/Adityaaop/Ads

1. INTRODUCTION & SELECTED OWASP CATEGORY

For this assignment, I selected **OWASP Top 10 (2021) Category A01:2021 – Broken Access Control**.

Broken Access Control currently holds the **#1 position** in the OWASP Top 10. Access control mechanisms enforce authorization policies so users cannot access resources or execute actions outside their intended permissions. When access controls are broken, attackers can view unauthorized files, steal server configurations, or access sensitive user records.

In this practical lab, I designed and implemented a dummy web application demonstrating **Path Traversal (CWE-22)** and **Insecure Direct Object Reference (CWE-639)** under **Improper Access Control (CWE-284)**. I conducted STRIDE threat modeling, calculated per-threat DREAD scores, implemented Defense-in-Depth mitigations in code, and verified the defenses using 17 automated unit test assertions.

2. COMPREHENSIVE MAPPING OF ALL 34 CWES TO OWASP A01:2021

According to the official OWASP Top 10:2021 taxonomy, Category A01:2021 maps directly to exactly **34 Common Weakness Enumerations (CWEs)**. Below is the complete table of all 34 mapped CWEs (note: CWE-73 is excluded as it formally belongs to A04:2021 - Insecure Design):

CWE ID	Weakness Name	Severity	Role in My Assignment / OWASP A01 Context
CWE-22	**Path Traversal (Improper Limitation of Pathname)**	High	**Primary Vulnerability Exploit (/download)**
CWE-23	Relative Path Traversal	Medium	Demonstrated using relative `../` sequences
CWE-35	Path Traversal: '.../...//'	Medium	Demonstrated via nested filter bypass `....//`
CWE-59	Improper Link Resolution Before File Access	Medium	Mitigated via `os.path.realpath` symlink checks
CWE-200	Exposure of Sensitive Information	Medium	Mitigated via safe 403 responses without path leaks
CWE-201	Insertion of Sensitive Information Into Sent Data	Medium	Mitigated by preventing credential leakage in HTTP body
CWE-219	Storage of File with Sensitive Data Under Web Root	High	Remediated by moving secrets outside web root
CWE-264	Permissions, Privileges, and Access Controls (Deprecated)	High	Overarching access control flaw category
CWE-275	Permission Issues	Medium	Solved by least-privilege folder filesystem permissions
CWE-276	Incorrect Default Permissions	High	Addressed via restrictive read-only permissions
CWE-284	**Improper Access Control**	High	**Core Architectural Flaw in routing**
CWE-285	Improper Authorization	High	Addressed through object-level permission verification
CWE-352	Cross-Site Request Forgery (CSRF)	High	Relevant to state-changing access operations
CWE-359	Exposure of Private Personal Information	Medium	Addressed by protecting student grade records
CWE-377	Insecure Temporary File	Medium	Mitigated by avoiding insecure temp file usage
CWE-402	Transmission of Private Resources into a New Sphere	Medium	Mitigated by strict canonical containment checks
CWE-425	Direct Request ('Forced Browsing')	Medium	Solved by tokenized indirect references
CWE-441	Unintended Proxy or Intermediary ('Confused Deputy')	Medium	Prevented by verifying client identity at origin
CWE-497	Exposure of Sensitive System Information	Low	Solved by generic error messages masking stack traces
CWE-538	Insertion of Sensitive Info in Externally-Accessible File	Medium	Remediated by isolating credentials from public docs
CWE-540	Inclusion of Sensitive Information in Source Code	Medium	Remediated via external environment config
CWE-548	Exposure of Information Through Directory Listing	Low	Mitigated by disabling directory indexing
CWE-551	Incorrect Behavior Order: Auth Before Canonicalization	High	Solved by canonicalizing path BEFORE access decision

CWE ID	Weakness Name	Severity	Role in My Assignment / OWASP A01 Context
CWE-552	Files or Directories Accessible to External Parties	Medium	Demonstrated via `.env` file exposure and fixed
CWE-566	Authorization Bypass Through User-Controlled SQL Key	High	Related to parameter-tampered primary keys in queries
CWE-601	URL Redirection to Untrusted Site ('Open Redirect')	Medium	Mitigated by rejecting external URI targets
CWE-639	**Authorization Bypass via User Key (IDOR)**	High	**Demonstrated & fixed via user ownership validation**
CWE-651	Exposure of WSDL File Containing Sensitive Info	Low	Specific to Web Services information exposure
CWE-668	Exposure of Resource to Wrong Sphere	High	Core concept in exposing server internal files
CWE-706	Use of Incorrectly-Resolved Name or Reference	Medium	Resolved using `os.path.realpath`
CWE-862	Missing Authorization	High	Addressed by adding mandatory authorization layer
CWE-863	Incorrect Authorization	High	Addressed by validating owner equals requester
CWE-913	Improper Control of Generation of Code	High	Prevented by strictly treating input as inert string
CWE-922	Insecure Storage of Sensitive Information	High	Remediated by isolating secrets from public directory

3. DUMMY APPLICATION SETUP & VULNERABILITY ANALYSIS

I created an academic portal in Python (Flask) with two setups:

- `vulnerable_app/` (Port 5000): Demonstrates CWE-22, CWE-35 (naive replace bypass), and CWE-639 (IDOR).
- `secure_app/` (Port 5001): Implements canonical checks, true per-user object-level authorization, structured audit logging, and 2MB file size limits.

```
# Vulnerable Code in vulnerable_app/app.py:
@app.route("/download")
def download_vulnerable():
    filename = request.args.get("file")
    # INSECURE: Direct concatenation allows ../ traversal
    file_path = os.path.join(PUBLIC_DOCS_DIR, filename)
    if not os.path.exists(file_path):
        return f"File not found: {filename}", 404
    return send_file(file_path)

# Vulnerable IDOR Endpoint (CWE-639):
@app.route("/api/document/")
def get_document_vulnerable(doc_id):
    # FLAW: No user ownership or authentication check
    doc = DOCUMENTS_DB.get(doc_id)
    return {"status": "success", "doc_id": doc_id, "data": doc}, 200
```

Failure of Naive Blacklisting (`.replace('../', '')`):

Developers often attempt to strip `../` with string replacement: `filtered = raw.replace("../", "")`. Because replacement runs only once, sending `....//confidential/admin_db_credentials.env` causes the inner `../` to be stripped, leaving behind `../` and executing traversal successfully with HTTP 200 OK.

4. THREAT MODELING (STRIDE)

I evaluated the document download feature against the six STRIDE threat categories with coded mitigations:

STRIDE Category	Threat in Application	Component	Potential Impact	Mitigation Implemented in Code
Spoofing	Attacker queries another user's document (e.g. `doc_03`) via IDOR	REST API parameter (`doc_id`)	Unauthorized access to confidential student records	Coded object-level authorization checking `doc['owner'] == current_user`.
Tampering	Attacker injects `../` or `....//` into filename	`/download?file=...`	Filesystem redirection to restricted directories	Regex allowlisting & canonical boundary checks (`os.path.realpath` / `commonpath`).
Repudiation	Attacker downloads sensitive files without leaving an audit trail	Access Logging	Inability to detect or trace data exfiltration	Structured JSON audit logging (`record_audit_log`) tracking IP, timestamp, user, and status.
Information Disclosure	Path traversal reads `.env` and database credentials	Local File System	Confidentiality loss; database credentials stolen	Canonical containment check via `os.path.commonpath`; secrets stored outside web root.
Denial of Service	Attacker requests infinite device streams (`/dev/urandom`) or huge files	OS File Descriptors	Worker process hangs; memory exhaustion	Enforced `os.path.isfile()` check and `MAX_FILE_SIZE = 2MB` constraint.
Elevation of Privilege	Attacker extracts `SECRET_KEY` to forge admin JWT/session cookies	Session Management	Administrative takeover of portal	Store secrets outside web root; block traversal; enforce server-side role validation.

5. DETAILED RISK ASSESSMENT (PER-THREAT DREAD MODEL)

To provide rigorous threat-specific quantification, I calculated DREAD scores for every STRIDE threat before and after mitigation: **Risk Score = (Damage + Reproducibility + Exploitability + Affected Users + Discoverability) / 5**

STRIDE Threat	Threat Scenario	Pre-Fix	Pre-Score	Post-Fix	Post-Score	Post-Mitigation Rationale
Spoofing (IDOR)	Cross-user IDOR access to Bob's confidential grades	7, 9, 9, 7, 8	8.0 / 10 (High)	2, 1, 2, 1, 2	1.6 / 10 (Low)	Ownership check rejects unauthorized requests with 403.
Tampering	Directory traversal via `../` or `....//`	9, 10, 10, 8, 9	9.2 / 10 (Critical)	1, 1, 1, 0, 2	1.0 / 10 (Negligible)	Canonical boundary check reliably blocks all traversal escapes.
Repudiation	Downloading sensitive files without audit trail	6, 8, 8, 6, 7	7.0 / 10 (High)	2, 1, 2, 1, 2	1.6 / 10 (Low)	Structured JSON audit log tracks all access attempts.
Information Disclosure	Leaking database passwords and server `.env`	10, 10, 10, 9, 9	9.6 / 10 (Critical)	1, 1, 1, 0, 2	1.0 / 10 (Negligible)	Secrets moved outside web root; canonical check blocks escape.
Denial of Service	Requesting device streams or oversized files	7, 8, 7, 8, 6	7.2 / 10 (High)	2, 2, 1, 1, 2	1.6 / 10 (Low)	Enforced `os.path.isfile()` and 2MB file size limit.
Elevation of Privilege	Stealing `SECRET_KEY` to forge admin cookies	10, 9, 8, 9, 8	8.8 / 10 (Critical)	2, 1, 2, 0, 2	1.4 / 10 (Low)	Root secrets protected; directory traversal sealed.
Composite Average	**Overall System Risk**	**8.2, 9.0, 8.7, 7.8, 7.8**	8.3 / 10 (CRITICAL)	**1.7, 1.2, 1.5, 0.5, 2.0**	1.37 / 10 (LOW RESIDUAL)	**Residual risk is fully acceptable; verified by 17 unit tests.**

6. THREAT AND EXPLOIT DEMONSTRATION

I executed proof-of-concept attacks against the vulnerable application using `scripts/exploit_demo.py` and `curl`:

1. **Normal Access:** `GET /download?file=syllabus_cs2341901.txt` → HTTP 200 OK.
2. **Stealing Database Credentials:** `GET /download?file=../confidential/admin_db_credentials.env` → HTTP 200 OK:

```
DB_HOST=10.0.12.44
DB_USER=ads_admin
DB_PASS=SuperSecretP@ssw0rd2026!
JWT_SECRET_KEY=9f83ab2018cf41a980e927164b38d0e7
```

3. **Stealing Root Server Secrets:** `GET /download?file=../../.env` → HTTP 200 OK (Exfiltrates production DB URIs and AWS keys).
4. **Bypassing Naive Filter:** `GET /download_naive?file=....//confidential/admin_db_credentials.env` → HTTP 200 OK.
5. **Exploiting IDOR (CWE-639):** `GET /api/document/doc_03` → HTTP 200 OK (Exfiltrates Bob's confidential grades without authentication).

7. MITIGATION IMPLEMENTATION & CODE COMPARISON

In `secure_app/app.py`, I implemented a Defense-in-Depth mitigation strategy:

- **Canonical Path Resolution:** Using `os.path.realpath()` and `os.path.commonpath()` to ensure files remain strictly inside `public_docs`.

- **Object-Level Authorization (IDOR Prevention):** Enforcing ownership check requiring `doc['owner'] == current_user` or admin.
- **Structured Audit Logging:** Function `record_audit_log` writes JSON events to `audit_access.log`.
- **DoS Protection:** Enforcing `os.path.isfile()` check and 2MB file size ceiling.

```
--- vulnerable_app/app.py (Vulnerable)
+++ secure_app/app.py (Hardened)
@@ -38,15 +48,45 @@
-@app.route("/download")
-def download_vulnerable():
-    filename = request.args.get("file")
-    # INSECURE: Direct concatenation allows ../ traversal
-    file_path = os.path.join(PUBLIC_DOCS_DIR, filename)
-    if not os.path.exists(file_path):
-        return f"File not found: {filename}", 404
-    return send_file(file_path)
+@app.route("/download_secure")
+def download_secure():
+    doc_id = request.args.get("doc_id")
+    current_user = request.headers.get("X-User-ID") or request.args.get("user_id", "aditya")
+    if doc_id not in AUTHORIZED_DOCUMENTS:
+        record_audit_log("UNKNOWN_RESOURCE_ACCESS", 403, user=current_user, resource=doc_id)
+        return "Access Denied: Invalid document identifier.", 403
+
+    doc = AUTHORIZED_DOCUMENTS[doc_id]
+    # DEFENSE 1: Object-level authorization check (CWE-639 IDOR mitigation)
+    if doc.get("owner") != current_user and current_user != "admin":
+        record_audit_log("IDOR_UNAUTHORIZED_ACCESS_BLOCKED", 403, user=current_user, resource=doc_id)
+        return "Access Denied: You do not have permission to access this resource.", 403
+
+    safe_name = doc["filename"]
+    full_path = os.path.join(PUBLIC_DOCS_DIR, safe_name)
+
+    # DEFENSE 2: Canonical boundary & DoS file size verification
+    safe, reason = is_safe_path(PUBLIC_DOCS_DIR, full_path)
+    if not safe:
+        record_audit_log("CANONICAL_BOUNDARY_VIOLATION", 403, user=current_user, resource=full_path)
+        return f"Security Violation: {reason}", 403
+
+    record_audit_log("FILE_DOWNLOAD_SUCCESS", 200, user=current_user, resource=doc_id)
+    return send_file(full_path)
```

8. AUTOMATED TEST SUITE VERIFICATION (17 TESTS)

I wrote an automated unittest suite in `scripts/test_suite.py` containing 17 test assertions:

```
======================================================================
RUNNING AUTOMATED SECURITY TEST SUITE (CS4021 – Assignment-1)
Student: Aditya Ranjan Kumar | CS-2341901 | 4CSE9
Instructor: Dr. Ajay Prasad
======================================================================

test_01_authorized_token_download (__main__.TestSecureHardenedApplication) ... ok
test_02_tampered_doc_id_traversal (__main__.TestSecureHardenedApplication) ... ok
test_03_tampered_doc_id_nonexistent (__main__.TestSecureHardenedApplication) ... ok
test_04_direct_name_path_traversal_blocked (__main__.TestSecureHardenedApplication) ... ok
test_05_direct_name_nested_traversal_blocked (__main__.TestSecureHardenedApplication) ... ok
test_06_direct_name_special_characters_blocked (__main__.TestSecureHardenedApplication) ... ok
test_07_direct_name_legitimate_download (__main__.TestSecureHardenedApplication) ... ok
test_08_idor_cross_user_access_blocked (__main__.TestSecureHardenedApplication) ... ok
test_09_idor_owner_permitted_access (__main__.TestSecureHardenedApplication) ... ok
test_10_idor_api_endpoint_blocked (__main__.TestSecureHardenedApplication) ... ok
test_11_structured_audit_logging_active (__main__.TestSecureHardenedApplication) ... ok
test_01_legitimate_download (__main__.TestVulnerableApplication) ... ok
test_02_path_traversal_confidential_env (__main__.TestVulnerableApplication) ... ok
test_03_path_traversal_root_env (__main__.TestVulnerableApplication) ... ok
test_04_naive_filter_bypass (__main__.TestVulnerableApplication) ... ok
test_05_missing_parameter (__main__.TestVulnerableApplication) ... ok
test_06_idor_unauthorized_record_access (__main__.TestVulnerableApplication) ... ok

----------------------------------------------------------------------
Ran 17 tests in 0.016s
OK
```

9. CONCLUSION

This lab demonstrates why Broken Access Control is ranked #1 in the OWASP Top 10. Direct file path concatenation without canonical path checks allows attackers to read sensitive files with simple `../` payloads. Naive string replacement like `.replace('../', '')` is ineffective against nested sequences like `..../`. The best defenses combine canonical path containment validation (`os.path.realpath` / `commonpath`) with strict object-level authorization checks.